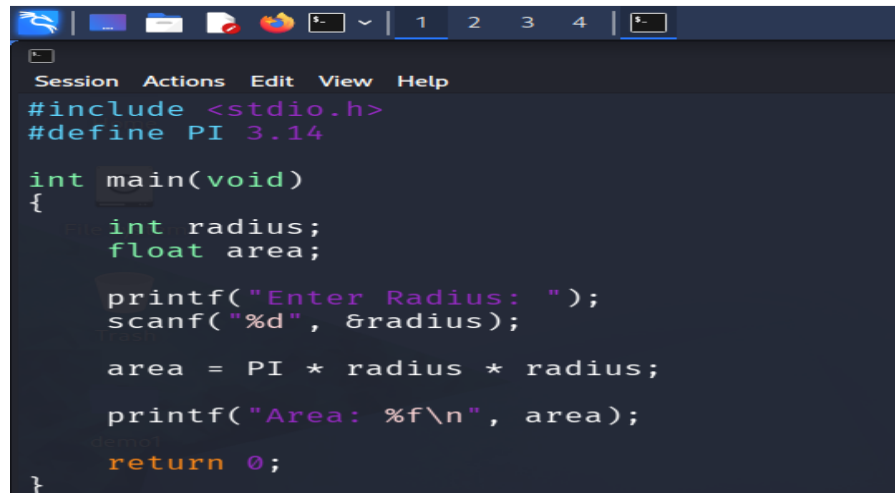


# EXPERIMENT – 2

## Stages of C Compilation and Area of a Circle

**Aim:** To study the different stages of compilation of a C program using the -save-temps option and execute a C program to calculate the area of a circle.

### 1.C Program – Area of a Circle

A screenshot of a code editor window showing a C program to calculate the area of a circle. The program includes `<stdio.h>`, defines `PI` as `3.14`, and has a `main` function that prompts for a radius, calculates the area using the formula `area = PI * radius * radius`, and prints the result.

```
Session Actions Edit View Help
#include <stdio.h>
#define PI 3.14

int main(void)
{
    int radius;
    float area;

    printf("Enter Radius: ");
    scanf("%d", &radius);

    area = PI * radius * radius;

    printf("Area: %f\n", area);

    return 0;
}
```

### 2. Compilation using -save-temps

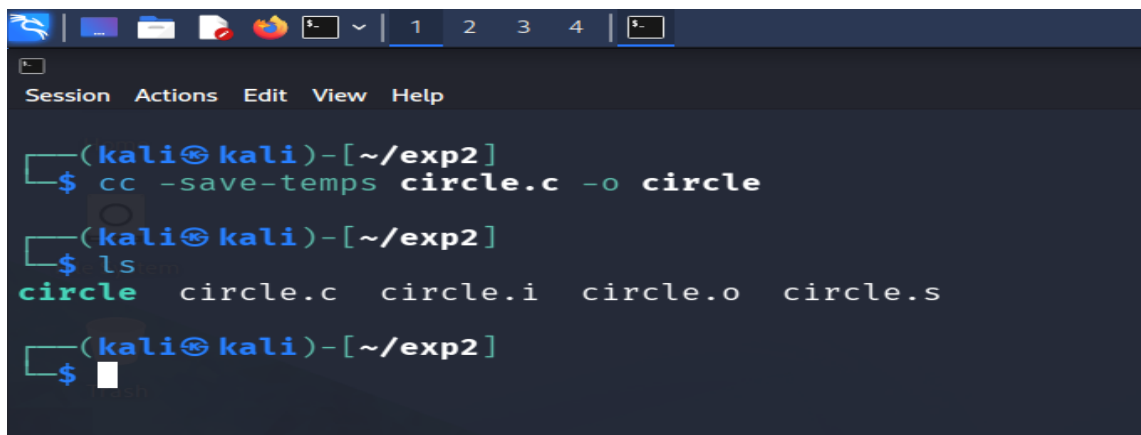
The following command compiles the program and saves the intermediate files produced during compilation:

```
cc -save-temps circle.c -o circle
```

```
ls
```

The generated files are:- **circle.c** → **circle.i** → **circle.s** → **circle.o** → **circle**

**Source Program** → **Preprocessing** → **Compilation** → **Assembly** → **Linking**

A screenshot of a terminal window showing the execution of compilation commands. The user runs `cc -save-temps circle.c -o circle` and then `ls`, which lists the generated files: `circle.c`, `circle.i`, `circle.o`, and `circle.s`.

```
Session Actions Edit View Help

(kali㉿kali)-[~/exp2]
$ cc -save-temps circle.c -o circle

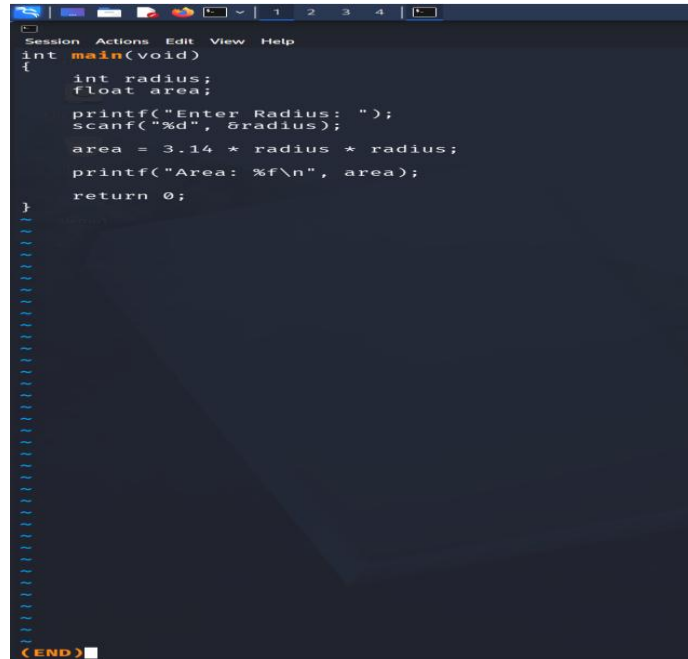
(kali㉿kali)-[~/exp2]
$ ls
circle circle.c circle.i circle.o circle.s

(kali㉿kali)-[~/exp2]
$
```

### 3. Preprocessing Stage – circle.c → circle.i

less circle.i

In this stage, the preprocessor processes preprocessor directives such as #include and #define. The resulting preprocessed source code is stored in the file circle.i.



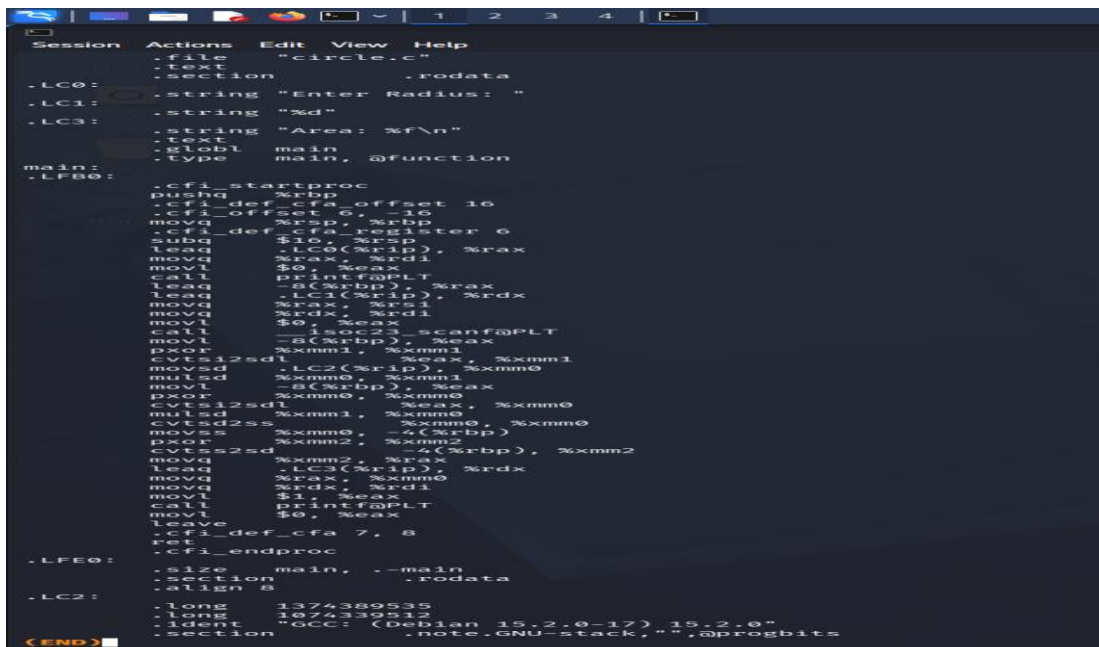
```
int main(void)
{
    int radius;
    float area;

    printf("Enter Radius: ");
    scanf("%d", &radius);
    area = 3.14 * radius * radius;
    printf("Area: %f\n", area);
    return 0;
}
```

### 4. Compilation Stage – circle.i → circle.s

less circle.s

In this stage, the compiler converts the preprocessed C program (circle.i) into assembly language. The generated assembly code is stored in the file circle.s.



```
.file "circle.c"
.text
.section .rodata
.LC0:
.string "Enter Radius: "
.LC1:
.string "%d"
.LC3:
.string "Area: %f\n"
.globl main
.type main, @function
main:
.LFB0:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 0, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
subq $16, %rsp
leaq .LC0(%rip), %rax
movq %rax, %rdi
movl $0, %eax
call printf@PLT
leaq .LC1(%rip), %rdx
movq %rax, %rsi
movq %rdx, %rdi
movl $0, %eax
call __isoc23_scanf@PLT
movl -8(%rbp), %eax
pxor %xmm1, %xmm1
cvtsi2sdl .LC2(%rip), %eax, %xmm1
mulsd %xmm0, %xmm1
movl -8(%rbp), %eax
pxor %xmm0, %xmm0
cvtsi2sdl %xmm1, %eax, %xmm0
mulsd %xmm0, %xmm0
cvtss2sd %xmm0, -4(%rbp), %xmm0
pxor %xmm2, %xmm2
cvtsd2sd %xmm0, -4(%rbp), %xmm2
movq %xmm2, %rax
leaq .LC3(%rip), %rdx
movq %rax, %xmm0
movq %rdx, %rdi
movl $1, %eax
call printf@PLT
movl $0, %eax
leave
.cfi_def_cfa 7, 8
ret
.cfi_endproc
.LFE0:
.size main, .-main
.section .rodata
.align 8
.LC2:
.long 1374389535
.long 1074339512
.long 0
.section .note.GNU-stack,"",@progbits
```

## 5. Assembly Stage – circle.s → circle.o

file circle.o

objdump -d circle.o

In this stage, the assembler converts the assembly-language program (circle.s) into machine-level object code. The generated object code is stored in the file circle.o.

```
(kali㉿kali)-[~/exp2]
$ file circle.o
circle.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped

(kali㉿kali)-[~/exp2]
$ objdump -d circle.o

circle.o:          file format elf64-x86-64

Disassembly of section .text:

0000000000000000 <main>:
0: 55                push    %rbp
1: 48 89 e5          mov     %rsp,%rbp
4: 48 83 ec 10       sub     $0x10,%rsp
8: 48 8d 05 00 00 00 lea     0x0(%rip),%rax        # f <main+0xf>
f: 48 89 c7          mov     %rax,%rdi
12: b8 00 00 00 00   mov     $0x0,%eax
17: e8 00 00 00 00   call   1c <main+0x1c>
1c: 48 8d 45 f8       lea     -0x8(%rbp),%rax
20: 48 8d 15 00 00 00 lea     0x0(%rip),%rdx        # 27 <main+0x27>
27: 48 89 c6          mov     %rax,%rsi
2a: 48 89 d7          mov     %rdx,%rdi
2d: b8 00 00 00 00   mov     $0x0,%eax
32: e8 00 00 00 00   call   37 <main+0x37>
37: 8b 45 f8          mov     -0x8(%rbp),%eax
3a: 66 0f ef c9       pxor    %xmm1,%xmm1
3e: f2 0f 2a c8       cvtsi2sd %eax,%xmm1
42: f2 0f 10 05 00 00 movsd   0x0(%rip),%xmm0        # 4a <main+0x4a>
49: 00
4a: f2 0f 59 c8       mulsd   %xmm0,%xmm1
4e: 8b 45 f8          mov     -0x8(%rbp),%eax
51: 66 0f ef c0       pxor    %xmm0,%xmm0
55: f2 0f 2a c0       cvtsi2sd %eax,%xmm0
59: f2 0f 59 c1       mulsd   %xmm1,%xmm0
5d: f2 0f 5a c0       cvtsd2ss %xmm0,%xmm0
61: f3 0f 11 45 fc   movss   %xmm0,-0x4(%rbp)
66: 66 0f ef d2       pxor    %xmm2,%xmm2
6a: f3 0f 5a 55 fc   cvtss2sd -0x4(%rbp),%xmm2
6f: 66 48 0f 7e d0   movq    %xmm2,%rax
74: 48 8d 15 00 00 00 lea     0x0(%rip),%rdx        # 7b <main+0x7b>
7b: 66 48 0f 6e c0   movq    %rax,%xmm0
80: 48 89 d7          mov     %rdx,%rdi
83: b8 01 00 00 00   mov     $0x1,%eax
88: e8 00 00 00 00   call   8d <main+0x8d>
8d: b8 00 00 00 00   mov     $0x0,%eax
92: c9              leave   %eax
93: c3              ret
```

## 6. Linking Stage – circle.o → circle

file circle

ls -l circle

In this stage, the linker combines the object file (circle.o) with the required library code and generates the final executable file named circle.

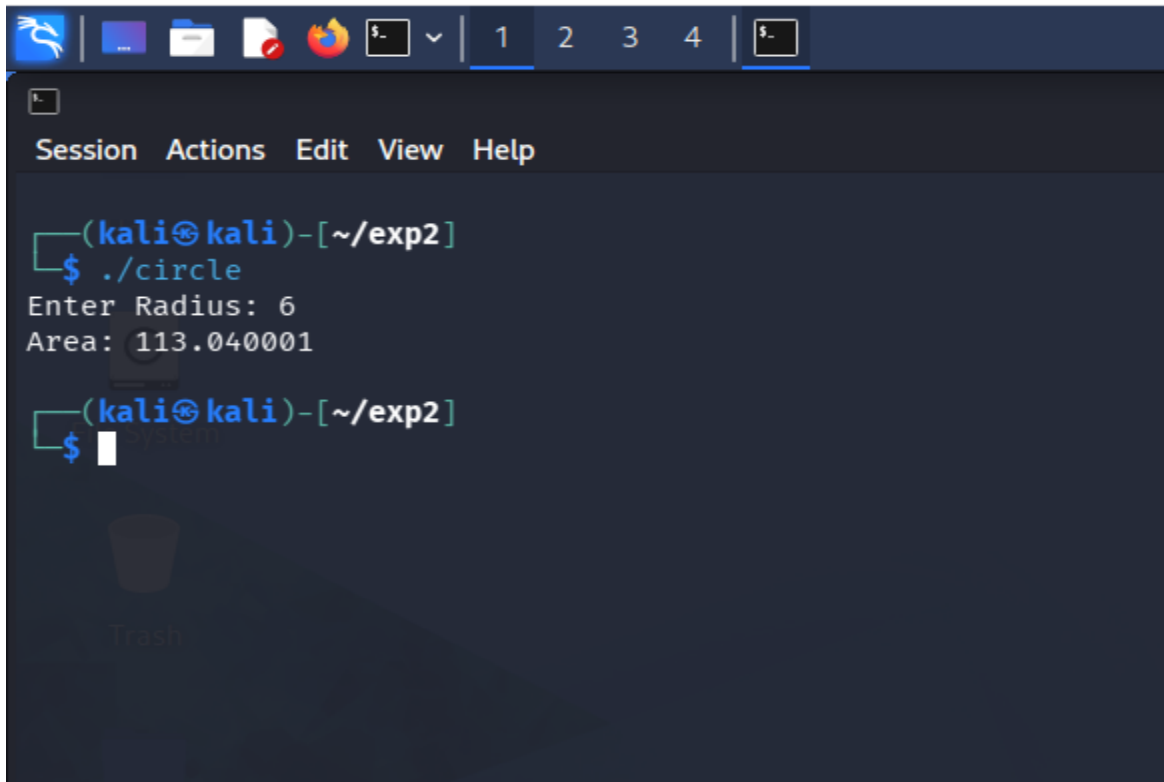
```
(kali㉿kali)-[~/exp2]
$ ls -l circle
-rwxrwxr-x 1 kali kali 16016 Aug 17 07:20 circle

(kali㉿kali)-[~/exp2]
$
```

## 7. Execution of the Program

The final executable is run using:-

`./circle`



The screenshot shows a terminal window with a dark blue background. The window has a title bar with icons for a terminal, a file manager, a document, and a web browser. Below the title bar is a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. The terminal content shows the following sequence of commands and output:

```
(kali㉿kali)-[~/exp2]  
$ ./circle  
Enter Radius: 6  
Area: 113.040001  
  
(kali㉿kali)-[~/exp2]  
$
```

The prompt is `(kali㉿kali)-[~/exp2]`. The first command is `./circle`, which prompts for the radius. The user enters `6`, and the program outputs `Area: 113.040001`. The second command is a blank line, and the prompt returns.

## 8. Result

The different stages of C compilation—preprocessing, compilation, assembly and linking—were successfully studied using the `-save-temps` option. The C program for calculating the area of a circle was compiled and executed successfully.